Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Logarithmic time and O(log n)

Today
Welcome to your next lesson: Logarithmic time and O(log n).

I'm preparing your personalized lesson based on what you've learned so far, what's coming up, and the best way for you to build your skills.

Step by step, you’re moving forward.

Imagine we’re playing the classic “guess my number” game 🎯

I’m thinking of a number between 1 and 100.

You guess 50. I say “too high”, so now you only care about 1–49.

Next you guess 25. If that’s too low, now you only care about 26–49.

Notice the pattern:

Each guess cuts the remaining options about in half. You never go 1, 2, 3, 4… one by one.

This “keep-halving” pattern is exactly the shape of logarithmic time, written as O(log n).

Quick check, just by intuition:

If you use this halving strategy on 1–100, about how many guesses do you usually need before you find the number?

You don’t need an exact number, just pick what feels closest:

“Around 5–7 guesses”
“Around 50 guesses”

Which one feels right to you?

5-7

Nice, that intuition is spot on 🎯

Needing around 5–7 guesses instead of 100 is the power of this keep-halving strategy —

that’s the heart of O(log n).

Let’s connect this to binary search for a second:

In binary search on a sorted list:

Each step: you check the middle element.
Then you throw away about half of the list.
Repeat: middle → throw away half → repeat…

So:

Logarithmic time (O(log n)) = algorithms where each step cuts the remaining work roughly in half.

Does that “binary search = keep-halving = O(log n)” connection make sense so far? 😊

If yes, I’ll show you a tiny Python example next to see how the steps grow.

Yes, that connection makes sense

Great 👍

Now I want to give you a visual feel for how slowly O(log n) grows compared to checking items one by one (O(n)).

Think about this table as we grow the list size:

List size (n)	Steps if you scan one by one (O(n))	Steps with binary search (O(log n))
8	~8	~3
16	~16	~4
32	~32	~5
64	~64	~6
128	~128	~7

Notice the pattern:

When n doubles (8→16→32→64→128),
Linear steps (O(n)) also roughly double
Logarithmic steps (O(log n)) just go +1, +1, +1… 😮
Does this pattern make sense:

“Double the data, linear work doubles, but logarithmic work just adds a tiny bit”?

If yes, I’ll show you a tiny Python snippet next that prints these step counts.

Yes, that pattern makes sense

Awesome 🍸 Let’s ground this with a tiny Python experiment.

Imagine we have a helper that pretends to do binary search and just counts how many steps it would take for different list sizes:

Python

```
def fake_binary_search_steps(n):
    steps = 0
```

```
    remaining = n
    while remaining > 1:
        remaining = remaining // 2   # cut the work in half
        steps += 1
    return steps

sizes = [8, 16, 32, 64, 128, 256, 512, 1024]

for n in sizes:
    print(n, fake_binary_search_steps(n))
```

Step by step:

remaining starts as n
Each loop: we do remaining // 2 → halve the remaining work
steps counts how many halvings until we're down to 1

You don't need to run this now; it's just to show that steps grow very slowly as n gets huge.

Quick intuition check:

As n goes from 8 to 1024, what happens to:

Linear steps O(n):
They go from 8 to 1024 (huge jump)

Log steps O(log n):
Do you expect them to:

Stay small (like under 20), or
Explode like the linear ones?

Just answer: "stay small" or "explode" and we'll build on that.

stay small

Exactly 💡 — they stay small. That's the magic of O(log n): even when data explodes, work grows very slowly.

Let's now place O(log n) on a simple "speed ladder" with the other classes you know:

O(1): constant time → same number of steps no matter how big n is
O(log n): logarithmic time → steps grow a little as n grows (keep-halving pattern)
O(n): linear time → steps grow in direct proportion to n

So in terms of speed (for large n):

Fastest: O(1)

In the middle: O(log n)

Slowest (of these three): O(n)

Make sense that O(log n) belongs between constant and linear in this ranking?

If yes, I'll give you a quick multiple-choice question to lock it in 🔓

Yes, that ranking makes sense

What do you think? Pick the order that feels right in the question below 👇

You're comparing three algorithms on a huge dataset.

Algorithm A takes O(1) time
Algorithm B takes O(log n) time
Algorithm C takes O(n) time
If we rank them from fastest to slowest for large n, which order is correct?

A.

A (O(1)), B (O(log n)), C (O(n))

B.

B (O(log n)), A (O(1)), C (O(n))

C.

C (O(n)), B (O(log n)), A (O(1))

Nice work Ron, you nailed it 🎉

For a huge n:

O(1) is fastest because it does the same tiny amount of work no matter how big the input is.
O(log n) is next: work grows slowly (just a few extra steps even when data explodes), thanks to the keep-halving pattern.
O(n) is slowest of these three because the work grows directly with n — double the data, about double the steps.

So your order

A (O(1)) → B (O(log n)) → C (O(n))

is exactly right ☑

Before we wrap this lesson, I want to double□check one thing in your own words:

How would you describe O(log n) to a friend who knows binary search but not Big□O yet?

You can keep it super short, like one sentence.

each step in the proccess cuts the workload in half until the answer is found or down to 1 item

That’s an excellent explanation, Ron 👌
You just captured the core idea of logarithmic time in plain language.

O(log n): each step cuts the remaining work about in half until you either find the answer or you’re down to 1 item.

You’ve hit all the key goals for this lesson:

You see O(log n) as the keep-halving pattern (like binary search).
You understand it grows much slower than O(n) as data grows.
You can correctly place it between O(1) and O(n) on the “speed ladder”.
You’re all set to move on to the next lesson 🚀
If you want, you can also throw me any quick follow-up question about O(log n) before you continue.